

This assignment is concerned with the mathematical description and composition of spatial transformations. You learn about and apply mathematical representations of rotations, such as rotation matrix, Euler angles, RPY angles, angle-axis and quaternions. The lab also covers homogeneous transforms between frames, which form the basis of kinematic models that relate robot joint to task space coordinates.

The following exercises are solved with Python in Jupyter notebooks and utilize NumPy and Matplotlib together with SpatialMath and the Robotics Toolbox for Python. The tasks involve the use of ROS 2 Jazzy on a Linux (Ubuntu) system which is also available in the Retina Pool. The robot platform used in this lab is the Elephant Robotics **myCobot 280** (myCobot 280 Pi).

Learning Objectives Spatial Transformations Lab

After completing the practical lab on *Spatial Transformations*, students will have a thorough understanding of the mathematical principles, representations, and computational methods used to describe and manipulate spatial transformations in robotics. They will be able to apply these concepts to model and control robotic manipulators, particularly in the context of transforming positions and orientations between different coordinate frames.

Students will learn how to represent rotations using various mathematical formulations, including rotation matrices, Euler angles, roll-pitch-yaw (RPY) angles, angle-axis representations, and unit quaternions. They will understand the advantages, limitations, and singularities associated with each representation and be able to convert between them. In particular, they will comprehend the relationships between intrinsic and extrinsic rotations and be able to compute and compose sequential rotations with respect to both fixed and moving frames.

A key objective is mastering the use of homogeneous transformation matrices, which combine rotation and translation into a unified framework. Students will learn how to construct, interpret, and apply these transformations to relate coordinate frames, perform forward kinematics, and transform vectors between frames in robotic applications. They will also gain practical experience composing multiple transformations and applying them to model the pose of robotic end-effectors, sensors, and objects in a workspace.

Furthermore, students will develop proficiency in using Python in Jupyter notebooks together with NumPy/Matplotlib, SpatialMath, and the Robotics Toolbox for Python to implement spatial transformations, visualize coordinate frame movements, and simulate kinematic structures such as rigid body trees. They will understand how translations and rotations interact in three-dimensional space and how these transformations are used to model robot links, joints, and motion chains.

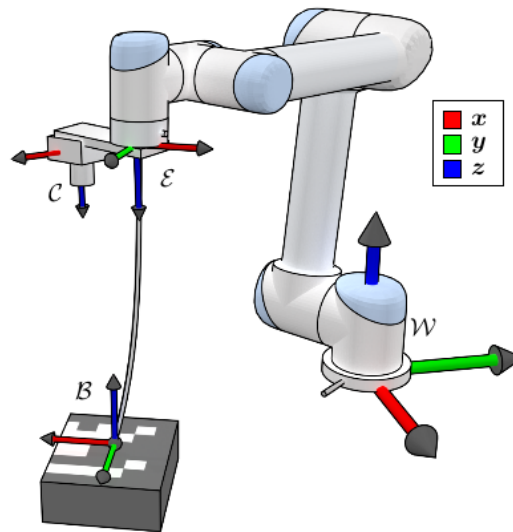
Students will also explore advanced rotation representations such as quaternions, learning how they overcome the limitations of Euler angles (e.g., gimbal lock) and how to use them for stable rotation composition and interpolation. They will implement spherical linear

interpolation (SLERP) for smooth rotational motion and compare it with interpolation based on Euler angles.

By the end of the lab, students will be able to:

- Mathematically describe and compute spatial transformations in 3D space.
- Represent, compose, and convert between different rotation parameterizations.
- Construct and apply homogeneous transformations for modeling robotic kinematics.
- Implement translations and rotations programmatically in Python.
- Model rigid body kinematic structures and interpret their spatial relationships.
- Use quaternions for rotation representation, composition, and interpolation.

These skills form a critical foundation for more advanced topics in robotics such as kinematic modeling, motion planning, control, and trajectory generation, and they are essential for understanding and designing robotic manipulation tasks in real-world applications.

Figure 1: Coordinate frames in robotics. ¹

Spatial Transformations

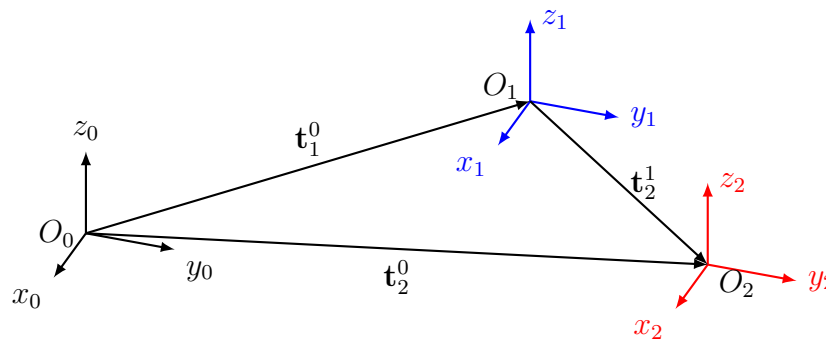
Robot kinematics establishes the transformations among various coordinate frames that capture the positions and orientations of end-effector, links, rigid bodies and cameras as shown in figure 1. In this scenario $\mathcal{W}$ denotes the robot base frame or world frame, $\mathcal{E}$ the robot end effector frame, $\mathcal{C}$ the camera frame and $\mathcal{B}$ the object frame. The mathematics of spatial transformations that describe rigid motions in Euclidean space is of fundamental importance to robotic manipulation.

The purpose of this assignment is to get familiar with representations of spatial transformations. A recommended reading is the section on position and orientation representation in the chapter on kinematics in the Springer Handbook of Robotics [?] or chapters 2.1–2.7 in the book by Siciliano et al [?]. Please carefully read these sections prior to the lab.

The mathematics of spatial transformations in Euclidean space is of fundamental importance to robotics. ROS 2 Jazzy and the Python robotics toolchain (NumPy + SpatialMath + Robotics Toolbox for Python) support multiple representations of rotations and spatial transformations.

- homogeneous transformations (4x4 matrix)
- rotation matrix (3x3 matrix)
- quaternions
- angle and vector orientation

¹Source: Torsteins Webpage Crane Control NTU

Figure 2: Translation of coordinate frames O_0, O_1, O_2 .

- Euler angles
- roll, pitch, yaw angles

Translations

A general rigid body motion is composed of a translation and a rotation. A translation in Euclidean space is represented by the components $\mathbf{t} = [t_x \ t_y \ t_z]$ of an ordinary 3D vector $\mathbf{t} = t_x \mathbf{x} + t_y \mathbf{y} + t_z \mathbf{z}$ in which $\mathbf{x}, \mathbf{y}, \mathbf{z}$ denote the unit vectors of the frame axes of an orthogonal frame $O - xyz$. Successive translations are obtained by mere vector addition $\mathbf{t}_0^2 = \mathbf{t}_0^1 + \mathbf{t}_1^2$. Figure 2 illustrates the translations $\mathbf{t}_0^1$ resp. $\mathbf{t}_1^2$ between frames $O_0 \rightarrow O_1$ resp. $O_1 \rightarrow O_2$.

In this lab, a translation is represented by the Cartesian translation vector `trvec`. In Python, we store it as a NumPy array of shape `(3,)` (units consistent with the task). For example, a translation by 3 units along the x-axis and 2.5 units along the z-axis is expressed as:

```
import numpy as np

trvec = np.array([3.0, 0.0, 2.5])
```

Successive translations are composed by mere vector algebra.

```
import numpy as np
import matplotlib.pyplot as plt

trvec1 = np.array([2.0, 1.0, 0.0])
trvec2 = np.array([0.0, 1.0, 1.0])
trvec12 = trvec1 + trvec2

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.quiver(0, 0, 0, trvec1[0], trvec1[1], trvec1[2], color="k")
ax.text(1, 0.5, 0, r"$\mathbf{trvec}_1$")
```

```

ax.quiver(trvec1[0], trvec1[1], trvec1[2], trvec2[0], trvec2[1], trvec2[2], color="k"
)
ax.text(2, 1.5, 0.5, r"$\text{trvec}_2$")
ax.quiver(0, 0, 0, trvec12[0], trvec12[1], trvec12[2], color="k")
ax.text(1, 1, 0.5, r"$\text{trvec}_{12}$")
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=30)
ax.grid(True)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
plt.show()

```

Rotation Matrices

Rotations are represented by 3×3 rotation matrices

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

which entries $\mathbf{R}_{ij}$ correspond to the direction cosines of the axes of frame $O - xyz$ w.r.t. frame $O' - x'y'z'$, namely

$$\begin{aligned}
r_{11} &= \mathbf{x}'^T \mathbf{x} = \cos(\angle \mathbf{x}', \mathbf{x}) \\
r_{12} &= \mathbf{y}'^T \mathbf{x} = \cos(\angle \mathbf{y}', \mathbf{x}) \\
r_{13} &= \mathbf{z}'^T \mathbf{x} = \cos(\angle \mathbf{z}', \mathbf{x}) \\
\dots &= \dots \\
r_{33} &= \mathbf{z}'^T \mathbf{z} = \cos(\angle \mathbf{z}', \mathbf{z})
\end{aligned}$$

in which $\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{x}', \mathbf{y}', \mathbf{z}'$ denote the unit vectors along the corresponding axes.

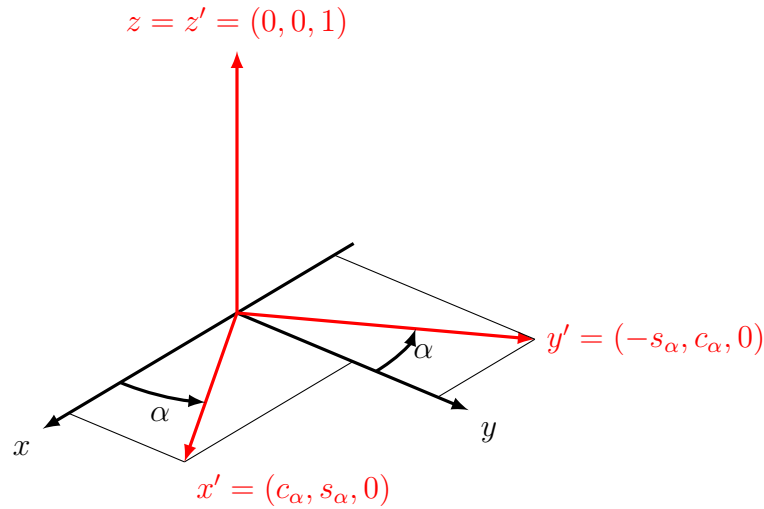
A rotation matrix $\mathbf{R}$ not only describes the relative orientation between frames but also performs a rotation of a vector in Euclidean space

$$\mathbf{p}' = \mathbf{R}\mathbf{p} \tag{1}$$

The matrix

$$\mathbf{R} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{2}$$

rotates vectors counter-clockwise through an angle θ about the z-axis of the Cartesian coordinate system. To perform the rotation using a rotation matrix $\mathbf{R}$, the position of a

Figure 3: Rotation by an angle α along the z-axis ²

point in space is represented by a 1-by-3 column vector $\mathbf{p} = [p_x \ p_y \ p_z]^T$, that contains the coordinates of the point w.r.t. the X-Y-Z-frame. A rotated vector is obtained by the matrix multiplication $\mathbf{R}\mathbf{p}$. Coordinate rotations are a natural way to express the orientation of a robot end effector, robot link or camera relative to a reference frame e.g. the robot base frame or a world frame. Once XYZ axes of a local frame are expressed numerically as three direction vectors w.r.t. a global world frame, they together comprise the columns of the rotation matrix $\mathbf{R}$ that transforms vectors in the reference frame into equivalent vectors expressed in the coordinates of the local frame. The inverse of rotation matrix coincides with its transpose, namely $\mathbf{R}^{-1} = \mathbf{R}^T$. Rotation matrices are square matrices that obey the orthogonality constraints: $\mathbf{R}^T = \mathbf{R}^{-1}$ and $\det(\mathbf{R}) = 1$.

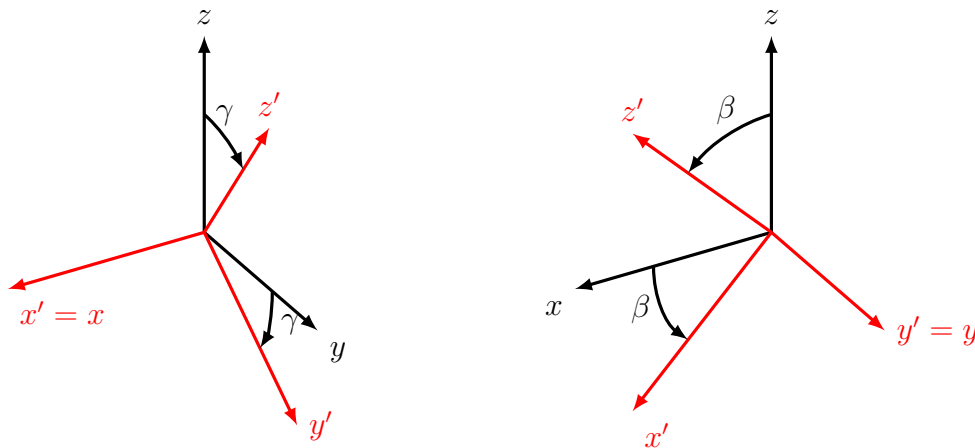
A basic or elemental rotation is a rotation along one of the axes of a Cartesian coordinate frame e.g. the z-axis by an angle α as shown in figure 3. The rotation is described by the rotation matrix

$$\mathbf{R}_z(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The rotation matrix

$$\mathbf{R}_z(\alpha) = [\mathbf{x}' \mid \mathbf{y}' \mid \mathbf{z}'] \quad (3)$$

is composed of the column vectors $\mathbf{x}' = (c_\alpha, s_\alpha, 0)^T$, $\mathbf{y}' = [-s_\alpha, c_\alpha, 0]^T$, $\mathbf{z}' = [0, 0, 1]^T$ shown in red in figure 3. The abbreviations c_α, s_α as short hand notations for $\cos \alpha, \sin \alpha$ are utilized in literature and throughout the course.

Figure 4: Rotations by γ, β along the x- and y-axis ³

Similarly, the elemental rotations along x- and y-axis by the angles γ, β shown in figure 4 are captured by the rotation matrices

$$\mathbf{R}_x(\gamma) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \gamma & -\sin \gamma \\ 0 & \sin \gamma & \cos \gamma \end{bmatrix}$$

$$\mathbf{R}_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix}$$

We denote a rotation matrix by `rotm`. It is a square, orthonormal matrix with determinant 1. In Python, it is represented as a 3×3 NumPy array. For example, elemental rotations of $\pi/2$ radians along x, y, z are generated with the commands:

```
import numpy as np

gamma = np.pi / 2
# elemental rotation about the x-axis
rotx = np.array(
    [
        [1.0, 0.0, 0.0],
        [0.0, np.cos(gamma), -np.sin(gamma)],
        [0.0, np.sin(gamma), np.cos(gamma)],
    ]
)
# elemental rotation about the y-axis
```

³source: RST

```

beta = np.pi / 4
rotmy = np.array(
    [
        [np.cos(beta), 0.0, np.sin(beta)],
        [0.0, 1.0, 0.0],
        [-np.sin(beta), 0.0, np.cos(beta)],
    ]
)
# elemental rotation about the z-axis
alpha = np.pi / 6
rotmz = np.array(
    [
        [np.cos(alpha), -np.sin(alpha), 0.0],
        [np.sin(alpha), np.cos(alpha), 0.0],
        [0.0, 0.0, 1.0],
    ]
)

```

Elemental rotations are more conveniently defined in angle axis notation and then converted to a rotation matrix

```

from scipy.spatial.transform import Rotation as R

# elemental rotation about the x-axis
rotmx = R.from_rotvec([gamma, 0.0, 0.0]).as_matrix()
# elemental rotation about the y-axis
rotmy = R.from_rotvec([0.0, beta, 0.0]).as_matrix()
# elemental rotation about the z-axis
rotmz = R.from_rotvec([0.0, 0.0, alpha]).as_matrix()

```

Elemental rotations can also be represented using SO(3) objects. In Python, the SpatialMath package provides the `S03` class, and SciPy's `scipy.spatial.transform.Rotation` can be used to generate rotation matrices. SO(3) rotations can be composed using multiplication (matrix product).

```

from scipy.spatial.transform import Rotation as R

rotmx = R.from_euler("x", gamma).as_matrix() # x-axis
rotmy = R.from_euler("y", beta).as_matrix()  # y-axis
rotmz = R.from_euler("z", alpha).as_matrix() # z-axis

```

Animation of Rotated Frames

The following code animates a rotation along the z-axis of the original frame $X_0 = [1, 0, 0]$, $Y_0 = [0, 1, 0]$, $Z_0 = [0, 0, 1]$. Figure 5 shows the final frame of the animation. The code updates the U, V, W components in the 3D quiver plot and relies on animated lines to incrementally draw the arcs of rotation.

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial.transform import Rotation as R

```

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.set_box_aspect([1, 1, 1])
ax.grid(True)
ax.set_xlim([-1.25, 1.25])
ax.set_ylim([-1.25, 1.25])
ax.set_zlim([-1.25, 1.25])

# Define the original frame (as column vectors)
O = np.array([0.0, 0.0, 0.0])
X0 = np.array([1.0, 0.0, 0.0])
Y0 = np.array([0.0, 1.0, 0.0])
Z0 = np.array([0.0, 0.0, 1.0])

alpha_f = np.pi / 4
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
ax.set_title(r"Rotation by  $\alpha$  in  $[0, \pi/4]$  along z-axis")

# Plot initial frame
hX = ax.quiver(0, 0, 0, X0[0], X0[1], X0[2], color="r", linewidth=2.5)
hY = ax.quiver(0, 0, 0, Y0[0], Y0[1], Y0[2], color="g", linewidth=2.5)
hZ = ax.quiver(0, 0, 0, Z0[0], Z0[1], Z0[2], color="b", linewidth=2.5)
ax.view_init(elev=30, azimuth=140)

# Plot initial frame dashed line
h1 = ax.quiver(0, 0, 0, X0[0], X0[1], X0[2], color="r", linewidth=2.5)
h2 = ax.quiver(0, 0, 0, Y0[0], Y0[1], Y0[2], color="g", linewidth=2.5)
h3 = ax.quiver(0, 0, 0, Z0[0], Z0[1], Z0[2], color="b", linewidth=2.5)

# Plot arc of rotation
scale = 0.5
arc_x = [scale * X0[0]]
arc_y = [scale * X0[1]]
arc_z = [scale * X0[2]]
h4, = ax.plot(arc_x, arc_y, arc_z, color="r", linewidth=2)
arc2_x = [scale * Y0[0]]
arc2_y = [scale * Y0[1]]
arc2_z = [scale * Y0[2]]
h5, = ax.plot(arc2_x, arc2_y, arc2_z, color="g", linewidth=2)
arc3_x = [scale * Z0[0]]
arc3_y = [scale * Z0[1]]
arc3_z = [scale * Z0[2]]
h6, = ax.plot(arc3_x, arc3_y, arc3_z, color="b", linewidth=2)

# Animate frame rotation
for alpha in np.arange(0.0, alpha_f + np.pi / 180.0, np.pi / 180.0):
    rotm = R.from_euler("z", alpha).as_matrix()
    X = rotm @ X0
    Y = rotm @ Y0
    Z = rotm @ Z0
```

```
hX.remove()
hY.remove()
hZ.remove()
hX = ax.quiver(0, 0, 0, X[0], X[1], X[2], color="r", linewidth=2.5)
hY = ax.quiver(0, 0, 0, Y[0], Y[1], Y[2], color="g", linewidth=2.5)
hZ = ax.quiver(0, 0, 0, Z[0], Z[1], Z[2], color="b", linewidth=2.5)

arc_x.append(scale * X[0])
arc_y.append(scale * X[1])
arc_z.append(scale * X[2])
h4.set_data(arc_x, arc_y)
h4.set_3d_properties(arc_z)

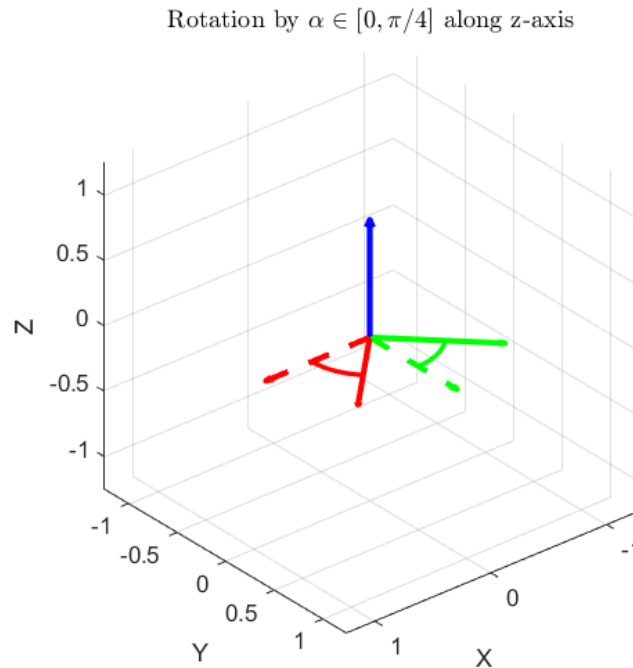
arc2_x.append(scale * Y[0])
arc2_y.append(scale * Y[1])
arc2_z.append(scale * Y[2])
h5.set_data(arc2_x, arc2_y)
h5.set_3d_properties(arc2_z)

arc3_x.append(scale * Z[0])
arc3_y.append(scale * Z[1])
arc3_z.append(scale * Z[2])
h6.set_data(arc3_x, arc3_y)
h6.set_3d_properties(arc3_z)

plt.pause(0.05)

# clear original frame and arcs
h1.remove()
h2.remove()
h3.remove()
h4.set_data([], [])
h4.set_3d_properties([])
h5.set_data([], [])
h5.set_3d_properties([])
h6.set_data([], [])
h6.set_3d_properties([])
plt.show()
```

⁴source: RST

Figure 5: Visualization of rotation along z-axis⁴

General Rotations

General rotations are obtained from these three elemental rotations by matrix post multiplication. The matrix product

$$\mathbf{R} = \mathbf{R}_z(\alpha) \mathbf{R}_y(\beta) \mathbf{R}_x(\gamma) \quad (4)$$

represents an intrinsic rotation in the order α , β and γ . The corresponding Python code is

```
import numpy as np
from scipy.spatial.transform import Rotation as R

alpha = np.pi / 6
beta = np.pi / 4
gamma = np.pi / 2
rotmz = R.from_euler("z", alpha).as_matrix()
rotmy = R.from_euler("y", beta).as_matrix()
rotx = R.from_euler("x", gamma).as_matrix()
rotrpyc = rotmz @ rotmy @ rotx
```

The same extrinsic rotation w.r.t. fixed frame in the order α , β and γ is

$$\mathbf{R} = \mathbf{R}_x(\gamma) \mathbf{R}_y(\beta) \mathbf{R}_z(\alpha) \quad (5)$$

```
rotropyf = rotmx @ rotmy @ rotmz
```

The difference between sequences of rotations becomes apparent by plotting the corresponding frames (see figure 6)

```
import numpy as np
import matplotlib.pyplot as plt

def plot_frame(ax, rotm, origin, length=1.0):
    basis = np.eye(3)
    colors = ["r", "g", "b"]
    for i in range(3):
        vec = rotm @ basis[:, i]
        ax.quiver(origin[0], origin[1], origin[2], vec[0], vec[1], vec[2], color=
        colors[i], length=length)

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
plot_frame(ax, np.eye(3), np.zeros(3), length=1.0)
plot_frame(ax, rottrpyc, np.zeros(3), length=1.0)
plot_frame(ax, rottrpyf, np.zeros(3), length=1.0)
ax.set_box_aspect([1, 1, 1])
plt.show()
```

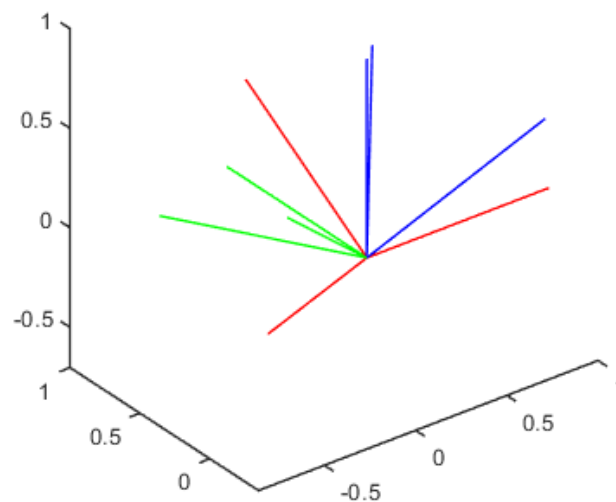
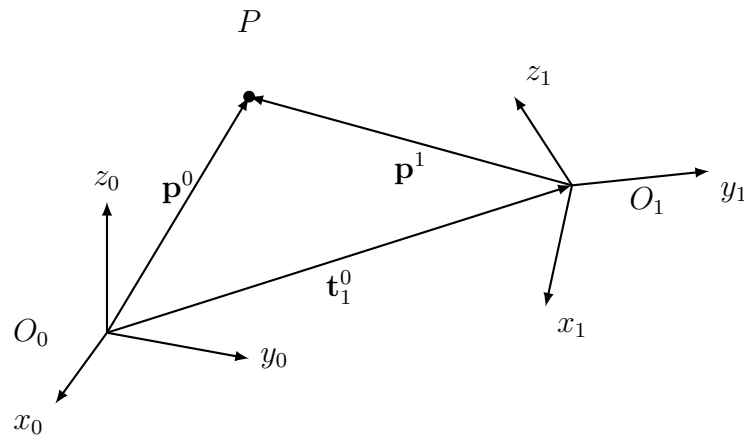


Figure 6: Original frame, frame `rottrpyf` and frame `rottrpyc` ⁵

⁵source: RST

Figure 7: Representation of a point P in frames O_0 and O_1 .

Rotation matrices not only describe the relative orientation between frames but also define the transformation of the coordinates of a vector expressed in one frame to the coordinates of the same vector expressed in another frame. Assume the frame O_1 is obtained from frame O_0 by applying a rotation $\mathbf{R}_1^0$ followed by a translation with a vector $\mathbf{t}_1^0$ as shown in figure 7. The coordinates of the point $\mathbf{p}^0$ with respect to frame $x_0y_0z_0$ are computed according to

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{p}^1 + \mathbf{t}_1^0 \quad (6)$$

```
import numpy as np
from scipy.spatial.transform import Rotation as R

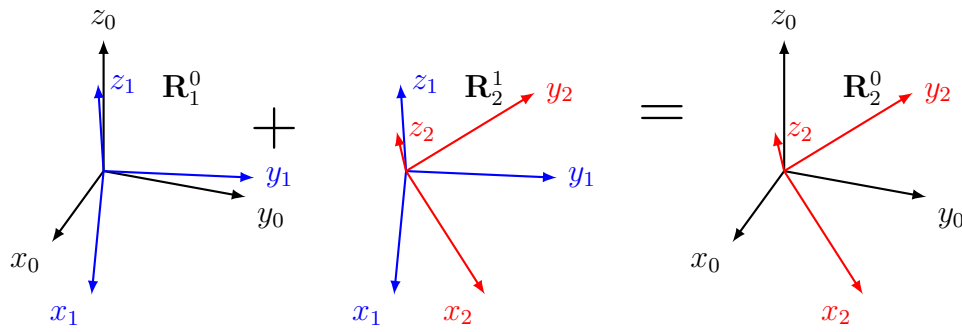
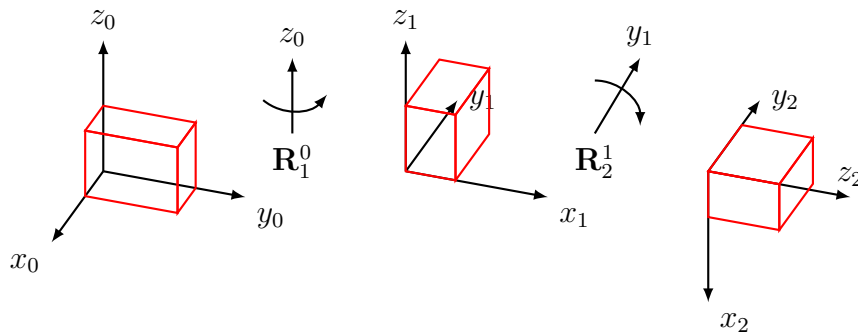
p1 = np.array([1.0, 1.0, 2.0])
trvec01 = np.array([1.0, 0.0, -1.0])
rotmx01 = R.from_euler("x", np.pi / 2).as_matrix()
p0 = rotmx01 @ p1 + trvec01
```

The following code first applies the translation $\mathbf{t}_1^0$ followed by a rotation $\mathbf{R}_1^0$ to the vector $\mathbf{p}^1$.

$$\mathbf{p}^0 = \mathbf{R}_1^0 (\mathbf{p}^1 + \mathbf{t}_1^0) \quad (7)$$

```
import numpy as np
from scipy.spatial.transform import Rotation as R

p1 = np.array([1.0, 1.0, 2.0])
trvec01 = np.array([1.0, 0.0, -1.0])
rotmx01 = R.from_euler("x", np.pi / 2).as_matrix()
p0 = rotmx01 @ (p1 + trvec01)
```

Figure 8: Consecutive rotations of frames O_0, O_1, O_2 .Figure 9: Consecutive rotations $\mathbf{R}_1^0, \mathbf{R}_2^1$ w.r.t. current frame.

Composition of Rotations

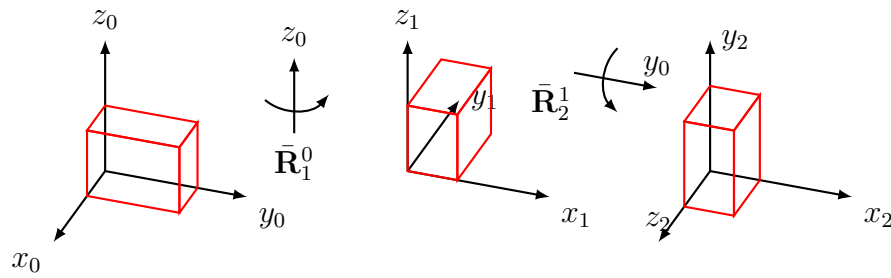
Consecutive rotations of frames are composed by multiplication of rotation. Figure 8 show a sequence of two rotations, a first rotation $\mathbf{R}_1^0$ from frame O_0 onto frame O_1 from a rotation followed by a second rotation $\mathbf{R}_2^1$ from frame O_1 onto frame O_2 . One distinguishes between consecutive rotations w.r.t. current and fixed frame. For current the consecutive rotation refer to the axis of the current frame, whereas for fixed frame all rotation refer to the axis of the initial frame. Figure 9 shows consecutive rotations along the z_0 of frame O_0 followed by a rotation along the y_1 -axis of the current frame O_1 .

If rotations are carried out w.r.t. the **current frame** O_1 **postmultiply** the matrices.

$$\mathbf{R}_2^0 = \mathbf{R}_1^0 \mathbf{R}_2^1 \quad (8)$$

The above matrix product corresponds to **intrinsic** rotations which are elemental rotations that occur about the axes of the rotating coordinate system $x_i y_i z_i$, which changes its orientation after each elemental rotation. An intrinsic composition uses rotations about **body-fixed axes** whose directions change in the reference frame after every rotation.

Figure 10 shows consecutive rotations along the z_0 of frame O_0 followed by a rotation along the y_0 -axis of the original (fixed) frame O_0 .

Figure 10: Consecutive rotations $\bar{\mathbf{R}}_1^0, \bar{\mathbf{R}}_2^1$ w.r.t. fixed frame.

If the second rotation $\bar{\mathbf{R}}_2^1$ is carried out w.r.t. the fixed frame $x_0y_0z_0$ **premultiply** the matrices (**extrinsic** rotation).

$$\bar{\mathbf{R}}_2^0 = \bar{\mathbf{R}}_2^1 \bar{\mathbf{R}}_1^0 \quad (9)$$

The bar above the rotation matrix denotes rotations that refer to a fixed frame.

Extrinsic rotations are elemental rotations that occur about the axes of the fixed coordinate system $x_0y_0z_0$. The XYZ system rotates, while $x_0y_0z_0$ is fixed. The corresponding extrinsic rotation about xyz is obtained by

$$\mathbf{R}_e = \mathbf{R}_x(\gamma) \mathbf{R}_y(\beta) \mathbf{R}_z(\alpha) \quad (10)$$

Such a transformation is called an extrinsic composition of rotations. In **extrinsic** rotations the rotations are performed about the **fixed principal axes** directions of the reference frame.

In the remainder of the course, in particular in the context of robot kinematics, it is assumed that rotations refer to current frame, if not stated otherwise.

The matrix $\mathbf{R}$ produces the desired effect only if they are used to pre-multiply column vectors. The vector pointing from the origin to $\mathbf{P}$ is either represented by a column vector $\mathbf{v}$ or a row vector $\mathbf{w}$. Rotation matrices either pre-multiply column vectors $\mathbf{R}\mathbf{v}$, or post-multiply row vectors $\mathbf{w}\mathbf{R}$. However, $\mathbf{R}\mathbf{v}$ produces a rotation in the opposite direction with respect to $\mathbf{w}\mathbf{R}$.

Start with tasks 1 to 5 in the .ipynb template.

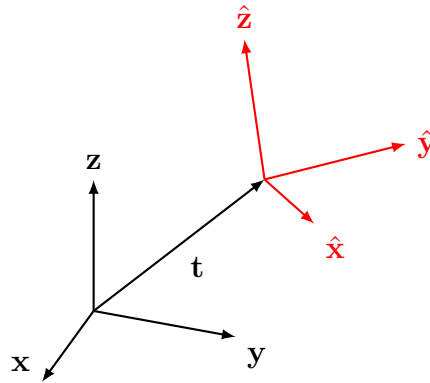


Figure 11: Homogeneous transform between frames $x - y - z$ and $\hat{x} - \hat{y} - \hat{z}$.⁶

Homogeneous Transformations

Homogeneous transformations are important as they provide the mathematical representation of direct (forward) kinematics. The relationships between frames, e.g. robot base frame and end effector frame, are defined by homogeneous transforms.

In the field of robotics there are many possible ways of representing positions and orientations, but the homogeneous transformation is most convenient as it combines the two operations of rotation and translation into a single matrix vector multiplication.

Homogeneous transformations are 4×4 matrices that describe the relationships between Cartesian coordinate frames in terms of translation and orientation. They are composed of a rotation matrix $\mathbf{R}$ and a translation vector $\mathbf{t}$.

$$\mathbf{H} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (11)$$

A homogeneous transformation matrix describes the relative pose between two frames in terms of translation and rotation as shown in figure 11. The vector $\mathbf{t}$ describes the translation between the origin of the two frames. The column vectors $\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}}$ that form the rotation matrix $\mathbf{R}$ denote the unit vectors of the rotated frame w.r.t. the reference frame $\mathbf{x}, \mathbf{y}, \mathbf{z}$.

$$\mathbf{H} = \begin{bmatrix} \hat{\mathbf{x}}_x & \hat{\mathbf{y}}_x & \hat{\mathbf{z}}_x & \mathbf{t}_x \\ \hat{\mathbf{x}}_y & \hat{\mathbf{y}}_y & \hat{\mathbf{z}}_y & \mathbf{t}_y \\ \hat{\mathbf{x}}_z & \hat{\mathbf{y}}_z & \hat{\mathbf{z}}_z & \mathbf{t}_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

⁶source: RST

Assume the frame $x_1 - y_1 - z_1$ is rotated w.r.t. frame $x_0 - y_0 - z_0$ by a rotation $\mathbf{R}_1^0$ and a translation with a vector $\mathbf{t}_1^0$ as shown in figure 7. The coordinates of the point $\mathbf{p}^0$ with respect to frame $x_0 y_0 z_0$ are computed according to

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{p}^1 + \mathbf{t}_1^0 \quad (12)$$

Assume another rotation and translation to obtain frame $x_2 y_2 z_2$ from frame $x_1 y_1 z_1$ by a rotation $\mathbf{R}_2^1$ followed by a translation $\mathbf{t}_2^1$.

$$\mathbf{p}^1 = \mathbf{R}_2^1 \mathbf{p}^2 + \mathbf{t}_2^1 \quad (13)$$

The composition of these consecutive rigid motions is given by:

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{R}_2^1 \mathbf{p}^2 + \mathbf{R}_1^0 \mathbf{t}_2^1 + \mathbf{t}_1^0 \quad (14)$$

thus

$$\begin{aligned} \mathbf{R}_2^0 &= \mathbf{R}_1^0 \mathbf{R}_2^1 \\ \mathbf{t}_2^0 &= \mathbf{R}_1^0 \mathbf{t}_2^1 + \mathbf{t}_1^0 \end{aligned}$$

This affine representation of rotation and translation has the drawback that rotational and translational components get mixed e.g. with terms such as $\mathbf{R}_1^0 \mathbf{t}_2^1$. A homogeneous representation replaces the matrix multiplication and vector addition in three dimensions by a matrix multiplication in four dimensions. For that purpose the Euclidean vectors $\mathbf{p}^0, \mathbf{p}^1$ are augmented by a fourth component (constant of 1) to obtain the homogeneous vectors:

$$\begin{aligned} \mathbf{P}^0 &= \begin{bmatrix} \mathbf{p}^0 \\ 1 \end{bmatrix} \\ \mathbf{P}^1 &= \begin{bmatrix} \mathbf{p}^1 \\ 1 \end{bmatrix} \end{aligned}$$

The transformation in equation (12) is equivalent to the homogeneous matrix transformation

$$\mathbf{P}^0 = \mathbf{H}_1^0 \mathbf{P}^1 \quad (15)$$

with the 4×4 homogeneous transformation

$$\mathbf{H}_1^0 = \begin{bmatrix} \mathbf{R}_1^0 & \mathbf{t}_1^0 \\ \mathbf{0} & 1 \end{bmatrix} \quad (16)$$

The same applies to

$$\mathbf{P}^1 = \mathbf{H}_2^1 \mathbf{P}^2 \quad (17)$$

The overall transformation in equation (14) is obtained from

$$\mathbf{P}^0 = \mathbf{H}_1^0 \mathbf{H}_2^1 \mathbf{P}^2 \quad (18)$$

such that

$$\mathbf{H}_2^0 = \mathbf{H}_1^0 \mathbf{H}_2^1 \quad (19)$$

is obtained from 4×4 matrix multiplication of the individual transformations.

We denote a homogeneous transformation matrix by `tform`. It combines a rotation and a translation in one 4×4 matrix. In Python, it is commonly represented as a 4×4 NumPy array, or as an `SE3` object from `SpatialMath`. For example, a rotation of angle `theta` around the x-axis followed by a translation of `dy` units along the y-axis is expressed as:

```
import numpy as np

theta = np.pi / 2
dy = 0.5
tform = np.array(
    [
        [1.0, 0.0, 0.0, 0.0],
        [0.0, np.cos(theta), -np.sin(theta), dy],
        [0.0, np.sin(theta), np.cos(theta), 0.0],
        [0.0, 0.0, 0.0, 1.0],
    ]
)
```

Represent homogeneous points as an $(N, 4)$ array of row vectors. Apply the transform with a transpose for column-vector multiplication, then transpose back. For example:

```
import numpy as np

points = np.random.rand(100, 3)
points_h = np.hstack([points, np.ones((points.shape[0], 1))])
tform_points_h = (tform @ points_h.T).T
```

In Python, conversions between representations can be done directly with NumPy (slicing/assembly) or with helper functions from `SpatialMath` / `Robotics Toolbox for Python`

```
import numpy as np

tform = np.eye(4)
tform[:3, 3] = trvec
```

generates the homogeneous transformation matrix `tform` that corresponds to a translation vector `trvec` with components $\mathbf{t} = [t_x, t_y, t_z]$.

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
import numpy as np

tform = np.eye(4)
tform[:3, :3] = rotm
```

generates the homogeneous transformation matrix `tform` ($\mathbf{H}$) that corresponds to a rotation matrix `rotm` ($\mathbf{R}$) with components r_{ij} .

$$\mathbf{H} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & 0 \\ r_{21} & r_{22} & r_{23} & 0 \\ r_{31} & r_{32} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Proceed with tasks 6 to 11 in the .ipynb template.

Rigid Body Tree Robot Model

Homogeneous transformations are ubiquitous in robotics and therefore in the Robotics Toolbox for Python. In particular they provide the basis for defining and utilizing robot direct and inverse kinematics as the corresponding frames are defined by homogeneous transforms. These examples are generic but aligned with ROS 2 Jazzy workflows and the myCobot 280 PI manipulator.

The rigid body tree model is a representation of a robot structure. It is useful to represent robots such as manipulators or other kinematic trees. The `ERobot`/`DHRobot` classes provide representations of manipulator kinematic models.

A rigid body tree is composed of rigid bodies (`Link`) that are connected via joints (`Joint`). Each rigid body has a joint that defines how that body moves relative to its parent in the tree. The method

```
from spatialmath import SE3

# fixed transform between parent and child link frames (model definition)
T_parent_child = SE3(tform)
```

specifies the transformation from one body to the next by setting the fixed transformation on each joint.

Bodies can be added, replaced, or removed from the rigid body tree model. You can also replace joints for specific bodies. The `ERobot/DHRobot` object maintains the relationships and updates the `Link` object properties to reflect this relationship. The transformations between different body frames are retrieved with

```
# forward kinematics: transform from base to a named link
T = robot.fkine(q, end="body") # returns SE3
```

Every rigid body tree has a base. The base defines the world coordinate frame and is the first attachment point for a rigid body. In Python, the base frame can be set via `robot.base` (an `SE3` transform) when needed.

The rigid body is the basic building block of a rigid body tree model and is created using

```
from roboticstoolbox import RevoluteDH

link = RevoluteDH(a=0.0, alpha=0.0, d=0.0, offset=0.0)
```

A rigid body, sometimes called a link, represents a solid body that is non-deformable.

Rigid body objects are added to a rigid body tree with multiple bodies. Rigid bodies possess parent and child relationships. The parent is the body that this rigid body is attached to, for example the robot base or some body upstream of the kinematic chain. The children are all the bodies attached to this body downstream from the base of the rigid body tree (see figure 12).

Each rigid body has a coordinate frame associated with them, and contains a `Joint` object.

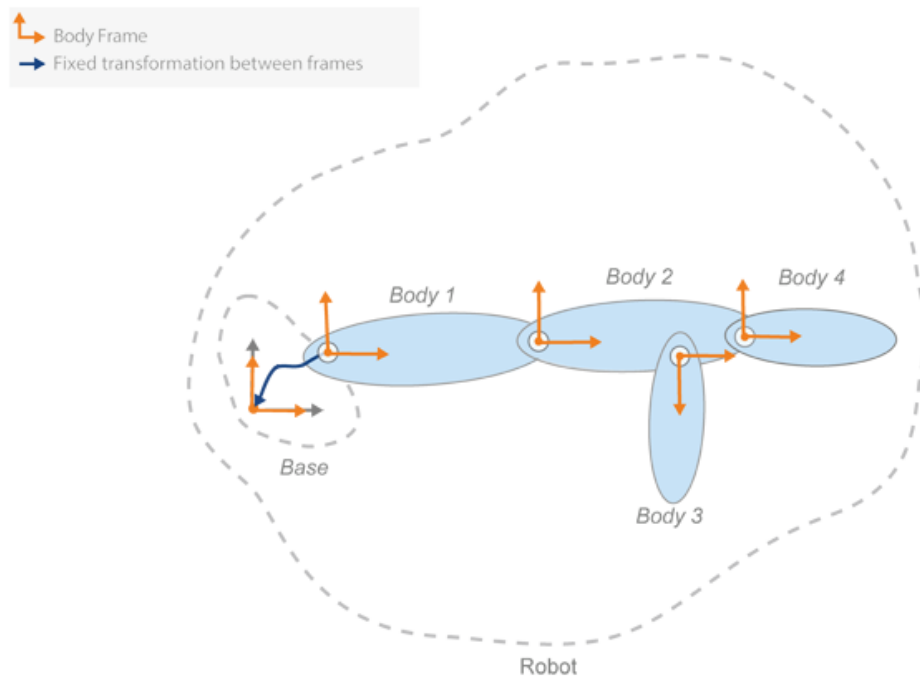
Each rigid body has one joint, which defines the motion of that rigid body relative to its parent. It is the attachment point that connects two rigid bodies in a robot model. To represent a single physical body with multiple joints or different axes of motion, use multiple `Link` objects.

The `Joint` class supports fixed, revolute, and prismatic joints as shown in figure 13.

These joints allow the following motion, depending on their type:

- `fixed` : No motion. Body is rigidly connected to its parent.
- `revolute` : Rotational motion only. Body rotates around this joint relative to its parent. Position limits define the minimum and maximum angular position in radians around the axis of motion.
- `prismatic` : Translational motion only. The body moves linearly relative to its parent along the axis of motion.

⁷source: documentation figure

Figure 12: Rigid body tree.⁷

This assignment only deals with fixed joints or entirely rigid bodies. The upcoming lab on robot kinematics introduces revolute and prismatic joints that form the kinematic chain of a manipulator.

Joints also have properties that define the fixed transformation between parent and child body coordinate frames. These properties are set by defining the link transform in the model, for example using

```
from spatialmath import SE3

# fixed transform between parent and child link frames (model definition)
T_parent_child = SE3(tform)
```

which represents the fixed parent-to-child transform. Figure 14 depicts what each transform signifies.

The following code generates a rigid body tree and attaches a rigid body with a fixed joint to the robot base.

```
from roboticstoolbox import DHRobot, RevoluteDH
from spatialmath import SE3
```

⁹source: documentation figure

¹¹source: documentation figure



Figure 13: The parent-to-joint transform defines where the joint of the child body is in relation to the parent body frame. The child-to-joint transform defines where the joint of the child body is in relation to the child body frame. ⁹

```
# Example: create a simple kinematic chain (robot model)
L1 = RevoluteDH(a=0.0, alpha=0.0, d=0.0, offset=0.0)
robot = DHRobot([L1], name="example")

# A fixed transform between the world frame and the robot base can be set via
robot.base = SE3(tform)
```

In the following you are supposed to generate a rigid body tree, with four bodies. Note, the underlying body frames coincide with the sequence of Denavit-Hartenberg elemental transformations as you will learn in the next assignment, namely a translation along z-axis by $d = 2$, rotation along z-axis by $\theta = \pi/4$, translation along the new x-axis by $a = 1$ and final rotation along the new x-axis by $\alpha = \pi/2$.

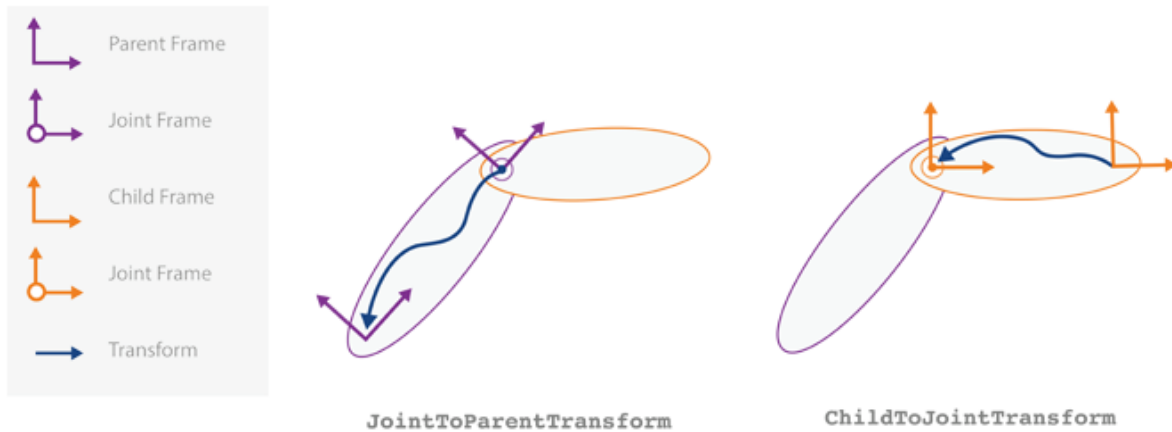


Figure 14: The parent-to-joint transform defines where the joint of the child body is in relation to the parent body frame. The child-to-joint transform defines where the joint of the child body is in relation to the child body frame. ¹¹

Proceed with tasks 12 to 23 in the .ipynb template.

Hint: The rigid body tree should resemble figure 15.

Euler Angles

A rotation matrix only possess three independent components as six out of nine elements are restricted by the orthogonality constraint. It seems plausible to seek a minimal representation of rotations in terms of merely three independent quantities, e.g. the angles of three consecutive elemental rotations. Euler angles compose the overall rotation into three successive rotations along designated axis as shown in figure 17. First rotate about the z-axis by the angle ϕ , then rotate about the current y-axis by the angle θ and finally rotate about the current z-axis by the angle ψ .

This particular order of rotation axes gives rise to the convention *ZYZ*-Euler angles. There are other combinations of axis, for e.g. *ZZZ*-Euler angles, but *ZYZ*-Euler angles are the most common ones. The overall rotation is obtained by postmultiplication of rotation matrices

$$\mathbf{R}_{\phi,\theta,\psi} = \mathbf{R}_{z,\phi} \mathbf{R}_{y,\theta} \mathbf{R}_{z,\psi} \quad (20)$$

Python tools support all proper and general sequences of Euler angles, namely "ZYX" (default), "YZY", "ZXY", "ZXZ", "YXY", "YZX", "YXZ", "YZY", "XYX", "XYZ", "XZX", "XZY". Knowing which axis order your code refers to is important for applying the rotation to points and in converting to other representations. The numeric representation

¹³source: RST

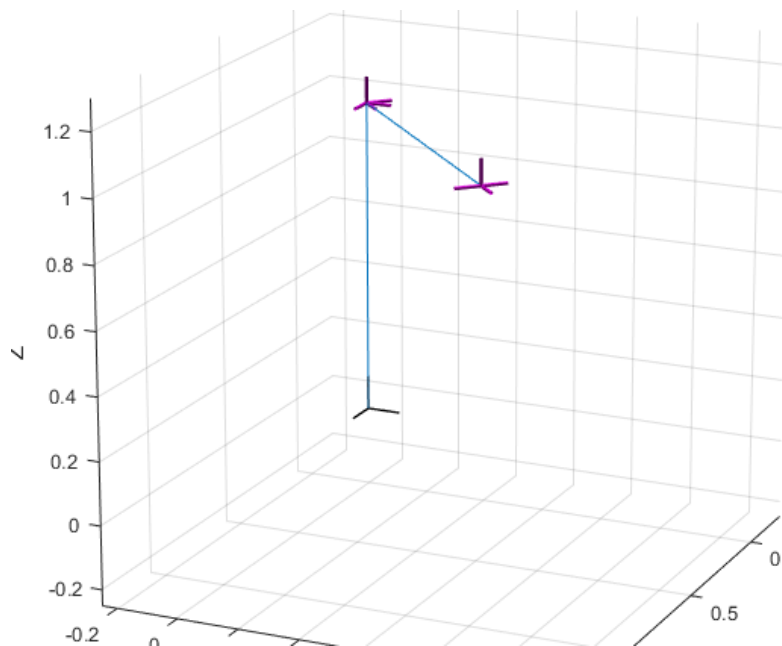


Figure 15: Rigid body tree for elemental Denavit-Hartenberg transforms

is an ordinary 1-by-3 vector of scalar angles. For example, a rotation along the y-axis by an angle π is expressed as:

```
import numpy as np
eul = np.array([0.0, np.pi, 0.0])
```

In Python, conversions between Euler angles and rotation matrices / homogeneous transforms can be performed with SciPy's `Rotation` (e.g., `from_euler`, `as_euler`) or with SpatialMath (S03/SE3). Always specify the axis sequence explicitly (e.g., 'ZYZ' or 'ZYX'), because different conventions exist.

```
import numpy as np
from scipy.spatial.transform import Rotation as R

phi = np.pi/6
theta = np.pi/4
psi = np.pi/3
rotmzyz = R.from_euler("ZYZ", [phi, theta, psi]).as_matrix()
```

Roll Pitch Yaw Angles

Roll, pitch and yaw angles are an alternative to ZYZ Euler angles, they denote rotations along the axes of a fixed frame rather than the current frame as shown in figure 18.

¹⁴source: RST

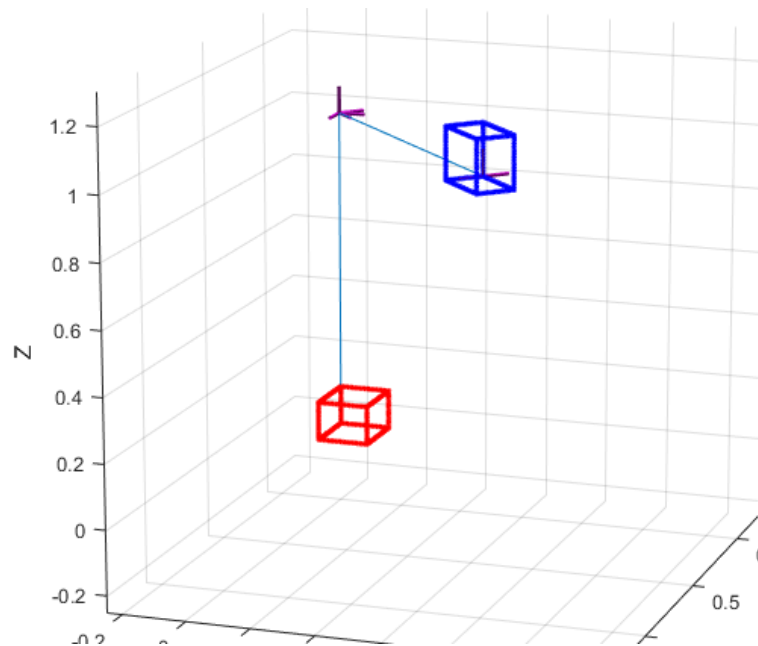


Figure 16: Cuboid w.r.t. base frame (red) and w.r.t. frame 'b4' (blue) (*Task 23*)

A rotation matrix $\mathbf{R}$ is described as a product of successive rotations about the principal coordinate axes x , y , z taken in a specific order. These rotations define the roll, pitch and yaw angles usually denoted by ϕ , ϑ and ψ as shown in figure 19. The overall rotation with respect to a fixed frame (extrinsic rotation) occurs by an angle ψ along the x -axis (roll), an angle ϑ along the y -axis (pitch) and an angle ϕ along the z -axis (yaw) resulting from the pre-multiplication:

$$\mathbf{R}_{\phi, \vartheta, \psi} = \mathbf{R}_{z, \phi} \mathbf{R}_{y, \vartheta} \mathbf{R}_{x, \psi} \quad (21)$$

Notice, that instead of roll-pitch-yaw (XYZ) relative to the fixed frames the transformation $\mathbf{R}_{\phi, \vartheta, \psi}$ can be interpreted as yaw-pitch-roll (ZYX) w.r.t. the current frame.

Roll Pitch Yaw angles are also denoted as Tait-Bryan angles. They are common in automotive and aerospace applications, so that zero degrees elevation represents the horizontal attitude. They are useful to capture the orientation of a vehicle or aircraft with respect to a fixed (global) world frame.

In Python, roll-pitch-yaw (RPY) angles can be created directly using SciPy's `Rotation` (or `SpatialMath`). As with Euler angles, you must be explicit about whether you are using an intrinsic (current-frame) or extrinsic (fixed-frame) convention. The intrinsic sequence 'ZYX' is commonly used for yaw-pitch-roll and corresponds to the extrinsic 'XYZ' convention (reversed multiplication order).

```
import numpy as np
from scipy.spatial.transform import Rotation as R
```

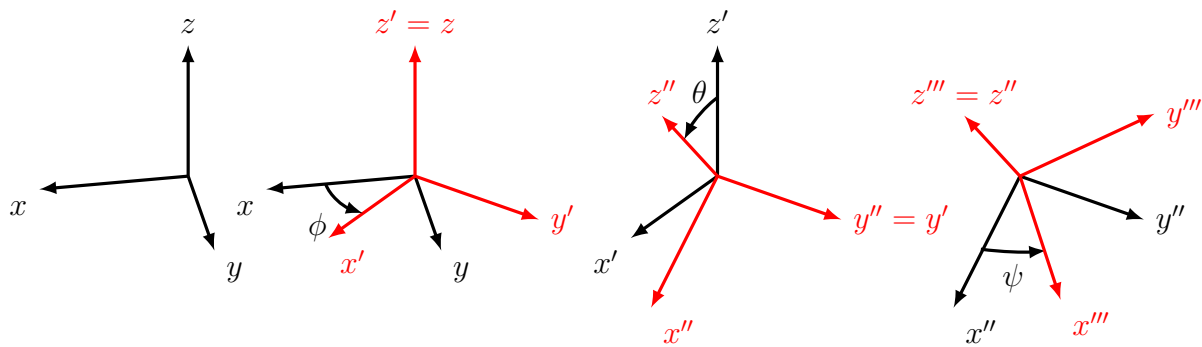


Figure 17: ZYZ - Euler angles, consecutive rotations w.r.t. current frame by ϕ along z-axis, θ along y' -axis and ψ along z'' -axis ¹³

```
phi = np.pi/6
theta = np.pi/4
psi = np.pi/3
rotmppy = R.from_euler("ZYX", [phi, theta, psi]).as_matrix()
```

Notice, that the order of the arguments of the Euler angles remains the same [**phi theta psi**]. Therefore 'ZYX' Euler angles w.r.t. current frame are equivalent to 'XYZ' roll pitch yaw angles w.r.t. fixed frame. For robotics manipulators 'ZYX' Euler angles are common whereas in mobile robotics 'XYZ' roll pitch yaw angles are common.

Axis - Angle

Euler's rotation theorem postulates that any rotation in 3D can be expressed as a single rotation about a designated axis which itself remains unchanged by the rotation. The magnitude of the angle is also unique, with its sign being determined by the sign of the rotation axis.

Rotations are not necessarily described by consecutive rotations w.r.t principal coordinate axes but instead in terms of a single rotation along an arbitrary axis in space. Let $\mathbf{r} = [r_x, r_y, r_z]^T$ be a unit vector that defines an axis of rotation expressed w.r.t. to the reference frame. The pair $\mathbf{r}, \vartheta$ with unit vector $\mathbf{r} = [r_x, r_y, r_z]^T$ and scalar ϑ defines a rotation by ϑ along $\mathbf{r}$. Since the axis is normalized, it has only two degrees of freedom. The angle adds the third degree of freedom to this rotation representation.

If the rotation angle ϑ is zero, the axis is not uniquely defined. Composition of rotations represented by Euler angles or axis-angle, is not straightforward as the axis vectors do not satisfy the law of vector addition. Typically, one converts from Euler angle / angle-axis to rotation matrix, performs the composition as a matrix product and then transforms back to Euler angle / angle-axis representation.

¹⁵source: RST

¹⁶source: RST

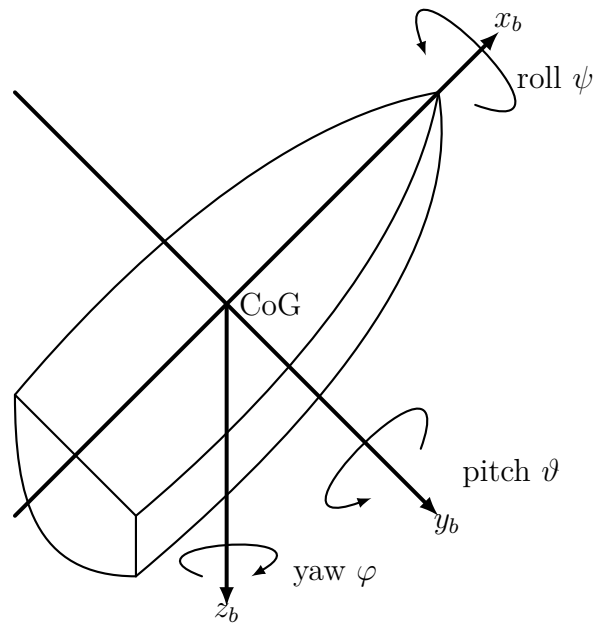


Figure 18: Roll Pitch Yaw angles ¹⁴

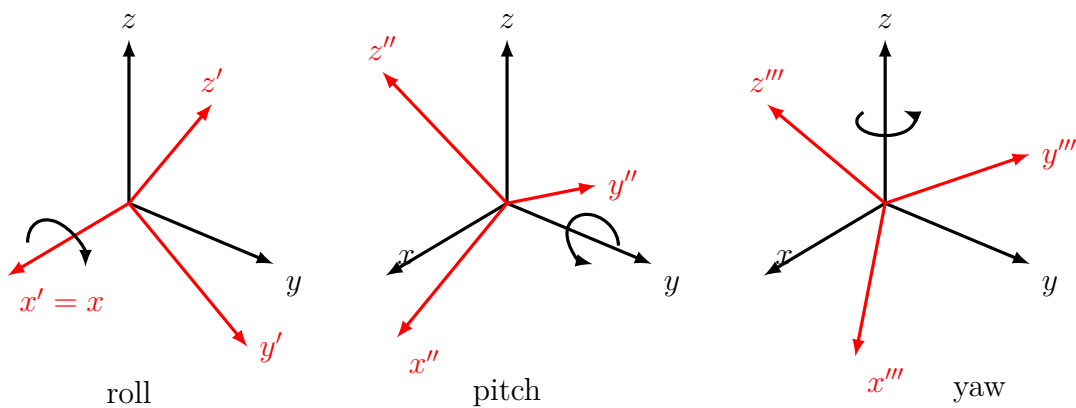


Figure 19: Roll Pitch Yaw angles ¹⁵

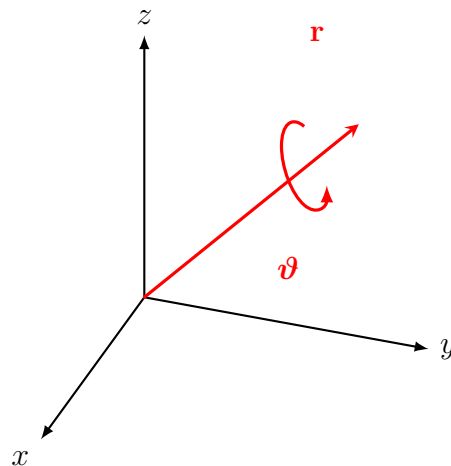


Figure 20: Rotation represented by an Euler axis $\mathbf{r}$ and angle ϑ ¹⁶

We denote the angle–axis representation by `axang`. It represents a scalar rotation of angle ϑ around a fixed unit axis $\mathbf{r}$. In Python, you can store it as a 1-by-4 vector `[rx ry rz theta]` (NumPy array), or use SciPy/SpatialMath helpers to convert to and from rotation matrices. For example, a rotation of $\pi/2$ radians around the y-axis: is defined by the vector `axang = [0 1 0 pi/2]`

Proceed with task 24 in the .ipynb template.

Animate Euler Angles and Roll Pitch Yaw Angles

Animate the sequence of rotations for 'ZYZ' Euler angles and 'XYZ' roll pitch yaw angles. Animate the rotating frame, the intermediates frames at the end of each single rotation and the arcs of rotation. Utilize the basic code for an elemental rotation along the z-axis as a starting point. Generate the rotation matrices of rotating frames with `Rotation.from_euler(...).as_matrix()` and apply them to the original unit vectors X_0, Y_0, Z_0 . For roll pitch yaw angles, use the reversed order (extrinsic composition of rotations).

Proceed with tasks 25 and 26 in the .ipynb template.

Representation Singularities

The task starts with the roll, pitch and yaw angles $\alpha = \pi/4, \beta = \pi/2$ and $\gamma = \pi/4$. The minimal representation of Euler and roll pitch yaw angles has the drawback of representation singularities, the so called gimbal lock. In the above example for roll, pitch yaw angles, the first z-axis coincides with the final x-axis, due to the rotation along the y-axis by $\pi/2$. The overall rotation along the common x- and z-axis by the angle $\alpha + \gamma$ becomes zero.

In case of Euler angles $\Phi = (\phi, \theta = 0, \phi)$ there is no rotation along the y-axis. As

a consequence, the final z''' -axis coincides with the first z -axis, and ϕ and ψ represent rotations about the same axis. The final orientation is obtained with a single rotation along z , by a total angle $\phi + \psi$.

Proceed with tasks 27 to 28 in the .ipynb template.

Unit Quaternions

Unit quaternions are another representation of orientation. They constitute a compromise between the advantages and disadvantages of rotation matrices and Euler angle sets. Compared to Euler angles they are simpler to compose and avoid the problem of gimbal lock. Compared to rotation matrices they are numerically more stable, e.g. when it comes to estimation of rotations, and more compact. A quaternion $\mathbf{q} \in \mathbb{H}$ with

$$\mathbf{q} = \eta + \epsilon_x \mathbf{i} + \epsilon_y \mathbf{j} + \epsilon_z \mathbf{k} \quad (22)$$

is composed of a **scalar part** $\text{Re}\{\mathbf{q}\} := \eta$ and an **imaginary part** (sometimes referred to as **vector part**) with three components $\text{Im}\{\mathbf{q}\} := \epsilon_x \mathbf{i} + \epsilon_y \mathbf{j} + \epsilon_z \mathbf{k}$. The imaginary part of a quaternion behaves like a vector in three dimension vector space, and the real part η behaves like a scalar in $\mathbb{R}$. The abstract symbols $\mathbf{i}, \mathbf{j}, \mathbf{k}$ are similar to the imaginary component of complex numbers and satisfy:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1 \quad (23)$$

For some operations it is useful to express the quaternion as a 4D vector, composed of the scalar and the vector part (notation $\mathbf{q}$ in boldface for the 4D vector representation):

$$\mathbf{q} = [\eta \ \boldsymbol{\epsilon}]^T = [\eta \ \epsilon_x \ \epsilon_y \ \epsilon_z]^T \quad (24)$$

Unit quaternions are constrained by

$$\|\mathbf{q}\| = \sqrt{\eta^2 + \boldsymbol{\epsilon}^T \boldsymbol{\epsilon}} = 1 \quad (25)$$

$$\iff \eta^2 + \boldsymbol{\epsilon}^T \boldsymbol{\epsilon} = 1 \quad (26)$$

The inverse of a quaternion is defined by

$$\mathbf{q}^{-1} = [\eta \ -\boldsymbol{\epsilon}]^T = [\eta \ -\epsilon_x \ -\epsilon_y \ -\epsilon_z]^T \quad (27)$$

The composition of rotations $q^{(1)}$ and $q^{(2)}$ (not squared!) is achieved by the quaternion product (Hamilton product)

$$\begin{aligned} (\eta^{(1)} + \epsilon_x^{(1)} \mathbf{i} + \epsilon_y^{(1)} \mathbf{j} + \epsilon_z^{(1)} \mathbf{k})(\eta^{(2)} + \epsilon_x^{(2)} \mathbf{i} + \epsilon_y^{(2)} \mathbf{j} + \epsilon_z^{(2)} \mathbf{k}) = \\ (\eta^{(1)} \eta^{(2)} - \epsilon_x^{(1)} \epsilon_x^{(2)} - \epsilon_y^{(1)} \epsilon_y^{(2)} - \epsilon_z^{(1)} \epsilon_z^{(2)}) + \\ (\eta^{(1)} \epsilon_x^{(2)} + \epsilon_x^{(1)} \eta^{(2)} + \epsilon_y^{(1)} \epsilon_z^{(2)} - \epsilon_z^{(1)} \epsilon_y^{(2)}) \mathbf{i} + \\ (\eta^{(1)} \epsilon_y^{(2)} - \epsilon_x^{(1)} \epsilon_z^{(2)} + \epsilon_y^{(1)} \eta^{(2)} + \epsilon_z^{(1)} \epsilon_x^{(2)}) \mathbf{j} + \\ (\eta^{(1)} \epsilon_z^{(2)} + \epsilon_x^{(1)} \epsilon_y^{(2)} - \epsilon_y^{(1)} \epsilon_x^{(2)} + \epsilon_z^{(1)} \eta^{(2)}) \mathbf{k} \end{aligned} \quad (28)$$

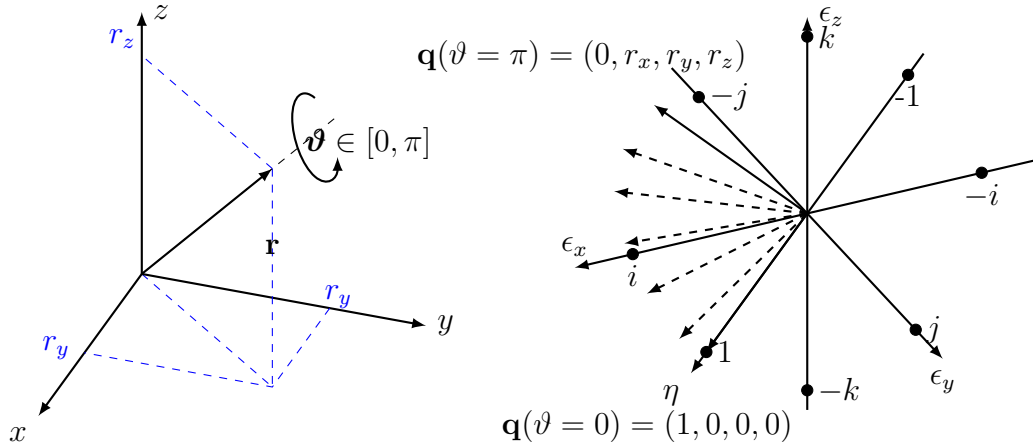


Figure 21: Angle axis representation $\mathbf{r}, \vartheta$ for $\vartheta \in [0, \pi]$ (left) and quaternion $\mathbf{q}(\vartheta) = (\eta, \epsilon_x, \epsilon_y, \epsilon_z)$ (right) .

A unit quaternion resembles the angle axis representation as it can be interpreted as a rotation about the axis denoted by $\frac{\epsilon}{\|\epsilon\|}$ by an angle θ .

$$q = e^{\frac{\theta}{2}(\epsilon_x \mathbf{i} + \epsilon_y \mathbf{j} + \epsilon_z \mathbf{k})} = \cos \frac{\theta}{2} + (\epsilon_x \mathbf{i} + \epsilon_y \mathbf{j} + \epsilon_z \mathbf{k}) \sin \frac{\theta}{2} \quad (29)$$

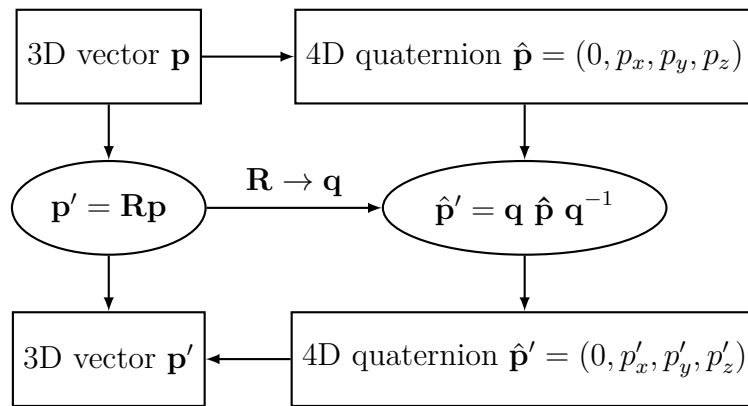
Figure 21 illustrates the relationship between angle axis representation in 3D space and the 4D quaternion space. On left $\mathbf{r}$ denotes the axis of rotation and the angle of rotation increases from $\vartheta = 0 \rightarrow \vartheta = \pi$. The corresponding unit quaternion evolves from $\mathbf{q}(\vartheta = 0) = (1, 0, 0, 0)$ to $\mathbf{q}(\vartheta = \pi) = (0, r_x, r_y, r_z)$. The 3D rotation axis becomes a 4D unit vector in the quaternion space, in which the scalar part $\eta = \cos \vartheta/2$ reflects the magnitude of rotation and the vector part $\epsilon_x, \epsilon_y, \epsilon_z$ denotes the rotation axis scaled by $\sin \vartheta/2$.

The rotation of an ordinary 3D vector $\mathbf{p} = [p_x, p_y, p_z]^T \in \mathbb{R}^3$ with a unit quaternion $\mathbf{q}$ is achieved by embedding the vector into $\mathbb{H}$ with $\hat{\mathbf{p}} = [0, p_x, p_y, p_z]^T$ and applying the conjugation of $\mathbf{p}$ by $\mathbf{q}$:

$$\hat{\mathbf{p}}' = \mathbf{q} \hat{\mathbf{p}} \mathbf{q}^{-1} \quad (30)$$

using the quaternion product defined in equation (28). The position vector $\mathbf{p}'$ denotes the coordinates of the point $\mathbf{p}$ after the rotation. The quaternion $\mathbf{p} = [\eta \ \epsilon]^T$ is composed of the vector part ϵ which is equal to the 3D vector $[p_x, p_y, p_z]^T$ and a zero real part $\eta = 0$. The vector part ϵ' of the resulting quaternion $\mathbf{p}'$ describes the desired 3D vector obtained by the quaternion product of $\mathbf{q}$, $\mathbf{p}$ and $\mathbf{q}^{-1}$.

Figure 22 illustrates the relationship between the ordinary Euclidian 3D space on the left and the 4D quaternion space on the right. The matrix-vector product $\mathbf{p}' = \mathbf{R} \mathbf{p}$ is replaced by the quaternion (Hamilton) product $\hat{\mathbf{p}}' = \mathbf{q} \hat{\mathbf{p}} \mathbf{q}^{-1}$.

Figure 22: Matrix vector product vs. quaternion product for rotation of point P .

SciPy's `Rotation` class stores unit quaternions internally and supports corresponding quaternion algebraic operations. Quaternions are instantiated from other rotation representations for example from rotation matrices

```
import numpy as np
from scipy.spatial.transform import Rotation as R

rotation_matrix = np.eye(3)
rot = R.from_matrix(rotation_matrix)
quat = rot.as_quat() # [x, y, z, w]
```

SciPy's `Rotation` applies active rotations to vectors by default. Instantiation from Euler angles

```
import numpy as np
from scipy.spatial.transform import Rotation as R

E = [np.pi / 6, np.pi / 4, np.pi / 3]
rot = R.from_euler("ZYX", E)
quat = rot.as_quat() # [x, y, z, w]
```

or from a rotation vector

```
import numpy as np
from scipy.spatial.transform import Rotation as R

rotation_vector = np.array([0.0, 0.0, np.pi / 4])
rot = R.from_rotvec(rotation_vector)
quat = rot.as_quat() # [x, y, z, w]
```

Rotation vectors $\mathbf{r}_{rotvec}$ are similar to angle-axis representation $(\vartheta, \mathbf{r}_{angleaxis})$, except that the magnitude of rotation ϑ is not a separate parameter but determines the length of the rotation axis vector.

$$\mathbf{r}_{rotvec} = \vartheta \mathbf{r}_{angleaxis} \quad (31)$$

vice versa

$$\mathbf{r}_{angleaxis} = \frac{\mathbf{r}_{rotxec}}{\|\mathbf{r}_{rotxec}\|} \quad \vartheta = \|\mathbf{r}_{rotxec}\|$$

The transformations from quaternion to Euler angle, rotation matrix and rotation vector are obtained by

```
from scipy.spatial.transform import Rotation as R

rot = R.from_quat([0.7071, 0.0, 0.0, 0.7071]) # SciPy expects [x, y, z, w]
euler_angles = rot.as_euler("ZYX")
rotation_matrix = rot.as_matrix()
rotation_vector = rot.as_rotvec()
```

The Rotation class supports quaternion algebra, in particular the Hamilton product

```
from scipy.spatial.transform import Rotation as R

Q1 = R.from_quat([0.0, 0.0, 0.0, 1.0])
Q2 = R.from_quat([0.0, 0.7071, 0.0, 0.7071])
Q1_times_Q2 = Q1 * Q2
```

and conjugation

```
Q1c = Q1.inv()
```

and quaternion power

```
from scipy.spatial.transform import Rotation as R

Q1square = R.from_rotvec(Q1.as_rotvec() * 2.0)
```

The command

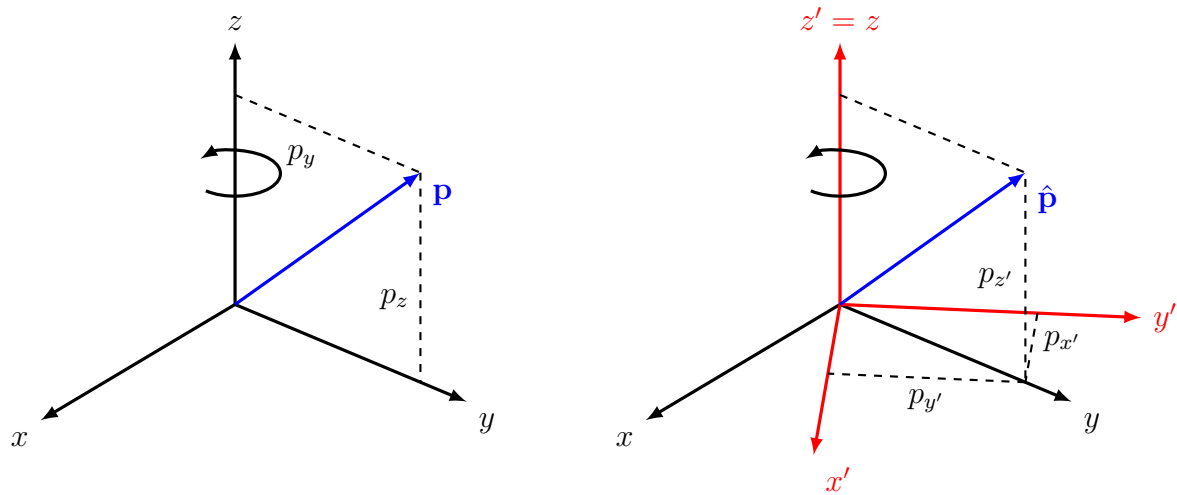
```
import numpy as np
from scipy.spatial.transform import Rotation as R

p = np.array([0.0, 0.7, 0.7])
quat = R.from_euler("XYZ", [[0.0, 0.0, np.pi / 4], [0.0, 0.0, -np.pi / 2]])
p_hat = quat.inv().apply(p)
```

rotates the frame of reference x-y-z onto the frame x'-y'-z' as shown in figure 23 and maps the Cartesian vector $\mathbf{p} = [p_x, p_y, p_z]^T$ in the x-y-z frame onto the same vector $\hat{\mathbf{p}} = [p_{x'}, p_{y'}, p_{z'}]^T$ in the x'-y'-z' frame using the quaternion quat.

The command

¹⁷Source: RST

Figure 23: Rotation of frame ¹⁷

```
import numpy as np
from scipy.spatial.transform import Rotation as R

p = np.array([0.5, 1.0, 1.0])
quat = R.from_euler("XYZ", [[0.0, 0.0, np.pi / 4], [0.0, 0.0, -np.pi / 2]])
p_hat = quat.apply(p)
```

rotates the vector $\mathbf{p} = [p_x, p_y, p_z]^T$ in a fixed frame of reference x-y-z onto the new vector with coordinates $\hat{\mathbf{p}} = [\hat{p}_x, \hat{p}_y, \hat{p}_z]^T$ w.r.t. the identical frame x-y-z as shown in figure 24.

Euler Angle Interpolation

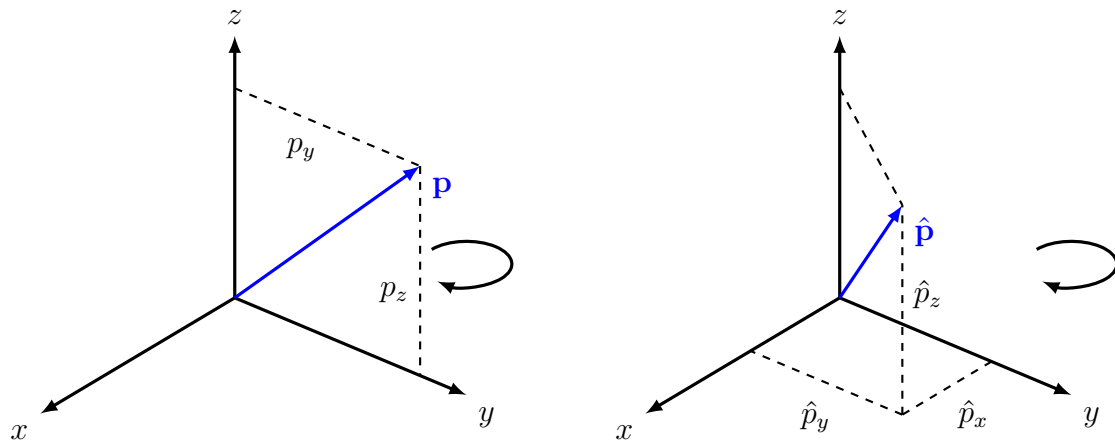
Interpolation of rotations via Euler angles are non-smooth. The vector moves on an inefficient arc and the axis of rotation is non-constant. The reason for the non-smooth trajectory is the simultaneous motion of two gimbals.

The figure 25 illustrates the trajectory of the vector $\mathbf{p} = [1, 0, 0]$ obtained from linear interpolation from the initial Euler angle $[\psi, \vartheta, \varphi]_i = [\pi/3, -\pi/2, 0]$ to the final Euler angle $[\psi, \vartheta, \varphi]_f = [-\pi/3, \pi/2 + \pi/20, 0]$. obtained from the code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial.transform import Rotation as R

path = np.arange(0.0, 1.0 + 0.05, 0.05)
p = np.array([1.0, 0.0, 0.0])
angles = np.vstack(
    [
        np.pi / 3 + (-path * 2 * np.pi / 3),
```

¹⁸Source: RST

Figure 24: Rotation of a point ¹⁸

```

        -np.pi / 2 + (path * np.pi * 1.05),
        0.0 + (path * 0.0),
    ]
).T
interpolated = R.from_euler("ZYX", angles)
r_peul = interpolated.apply(p)

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot(r_peul[:, 0], r_peul[:, 1], r_peul[:, 2], "r-d")
ax.quiver(
    np.zeros(len(path)),
    np.zeros(len(path)),
    np.zeros(len(path)),
    r_peul[:, 0],
    r_peul[:, 1],
    r_peul[:, 2],
    color="r",
)
plt.show()

```

Lerp and Slerp

Interpolation rotation with Euler angles is suboptimal. The motion that results from simply interpolating between the Euler angle of both frames is often non-smooth, jerky and ill-defined. Lerp is the abbreviation for a linear interpolation among two vectors

$$\text{lerp}(\mathbf{p}_0, \mathbf{p}_1, t) = (1 - t)\mathbf{p}_0 + t\mathbf{p}_1$$

as shown in the l.h.s. of figure 28.

Slerp stands for spherical linear interpolation with the purpose of smoothly interpolating 3D rotations. It allows a smooth rotation of a vector along a great circle arc by an angle

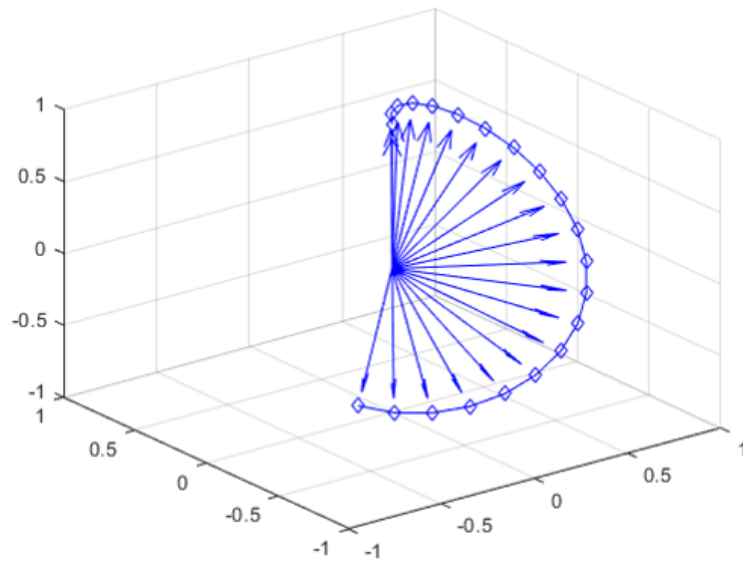


Figure 25: Euler angle interpolation of rotated frames on vector $\mathbf{p} = [1, 0, 0]$.

Ω between orientations $\mathbf{p}_0$ and $\mathbf{p}_1$ with the interpolation parameter $t \in [0, 1]$ as shown in the r.h.s. of figure 28.

$$\text{slerp}(\mathbf{p}_0, \mathbf{p}_1, t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} \mathbf{p}_0 + \frac{\sin(t\Omega)}{\sin \Omega} \mathbf{p}_1$$

Quaternion Slerp

Slerp is applied to unit quaternions such that the quaternion path maps to a 3D rotation along a great circle arc as shown in figure 29. The effect is a uniformly interpolated rotation around a fixed rotation axis. Given an initial quaternion q_0 and a final quaternion q_1 the interpolation over the interval $t = [0, 1]$ is given by the Hamilton product.

$$\mathbf{q}(t) = \mathbf{q}_0(\mathbf{q}_0^{-1}\mathbf{q}_1)^t \quad (32)$$

such that $\mathbf{q}(0) = \mathbf{q}_0$ and $\mathbf{q}(1) = \mathbf{q}_1$. Slerp is useful for interpolating rotations for example in path and trajectory planning between a start and a goal pose in workspace.

The function

```
import numpy as np
from scipy.spatial.transform import Rotation as R, Slerp

q0 = np.array([0.0, 0.0, 0.0, 1.0])
q1 = np.array([0.0, 0.0, 0.7071, 0.7071])
key_times = [0.0, 1.0]
```

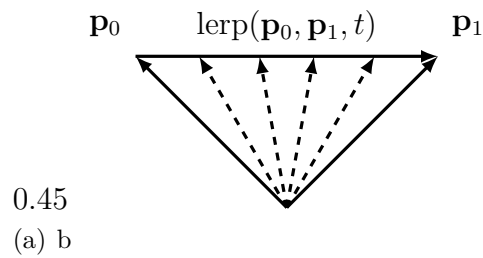


Figure 26: Lerp

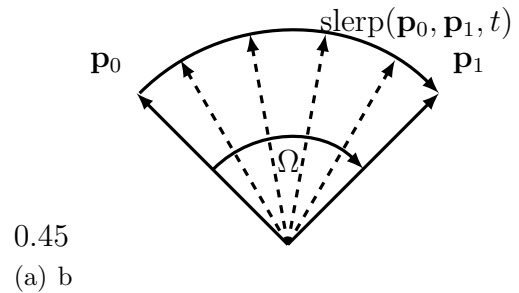


Figure 27: Slerp

Figure 28: Lerp and Slerp.

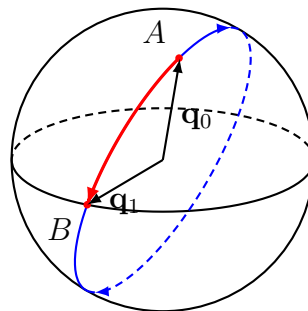


Figure 29: Quaternion Slerp along great circle arc.

```
key_rots = R.from_quat(np.vstack([q0, q1])) # q0, q1 as [x, y, z, w] (SciPy order)
slerp = Slerp(key_times, key_rots)
t = np.linspace(0.0, 1.0, 50)
qs = slerp(t)
```

calculates the quaternion spherical linear interpolation according to equation 32 and interpolates between q_0 and q_1 by the interpolation points in t .

Proceed with tasks 29 to 33 in the .ipynb template.

Notes:

- Compare the quaternion interpolation of task 30 with the Euler angle interpolation in figure 25.

Competences

After successfully completing the Spatial Transformations lab, students will have acquired the following competences:

- They can mathematically describe and compute spatial transformations in three-dimensional space.
- They are able to represent, convert, and compose different rotation parameterizations, including rotation matrices, Euler angles, roll-pitch-yaw angles, angle-axis representations, and unit quaternions.
- They understand the relationships between intrinsic and extrinsic rotations and can compute sequential rotations with respect to fixed and moving frames.
- They can construct, interpret, and apply homogeneous transformation matrices to relate coordinate frames, perform forward kinematics, and transform vectors between frames.
- They are capable of implementing translations, rotations, and compositions of transformations programmatically using Python in Jupyter notebooks together with SpatialMath and the Robotics Toolbox for Python.
- They can model rigid-body kinematic structures (e.g., robotic manipulators) and analyze spatial relationships between robot components, sensors, and objects.
- They are proficient in using quaternions for stable rotation representation, composition, and interpolation, and can implement spherical linear interpolation (SLERP) for smooth rotational motion.
- They can evaluate and mitigate representation singularities (e.g., gimbal lock) and understand the advantages and limitations of different rotation representations.
- They are able to apply spatial transformation principles to model and simulate robotic manipulators, including the pose estimation of end-effectors and coordinate transformations between different frames.
- They have the ability to integrate spatial transformations into kinematic modeling, motion planning, and control tasks in robotics applications.